

Rapport Compte Rendu Bureau d'Etudes Groupe 6 Développement d'un outil de communication avec un analyseur de gaz

Valerio ANGIONE , Lenny SABINE , Vu Bao Chau DANG and
Elijah PEYRAT

Université Toulouse III Paul Sabatier
Licence Informatique
Encadrant : Gilles ATHIER (LAERO)

June 08, 2026

SOMMAIRE

1.	Rappel contexte et exigences	5
1.1.	Contexte du projet	5
1.2.	Objectif et exigences	5
2.	Partie technique	6
2.1.	Protocole de communication	6
2.1.1.	C-link	6
2.1.2.	Modbus	6
2.2.	Support de connexion	6
2.2.1.	Port serie	6
2.2.2.	Ethernet	7
2.3.	Communication avec l'analyseur	7
2.3.1.	Syntaxe des requêtes	7
2.3.2.	Commandes utilisées	7
2.4.	Récupération des données	8
2.4.1.	Forme de la réponse et sauvegarde CSV	8
2.4.2.	Désynchronisation	8
2.5.	Interface graphique	9
2.5.1.	Exigences	9
2.5.2.	Choix des bibliothèques	10
2.5.3.	Implémentation	10
2.6.	La communication entre backend et frontend	11
2.6.1.	Version 1 : Communication via CSV	11
2.6.1.1.	Vue l'ensemble de l'architecture via CSV	11
2.6.1.2.	Des avantages et désavantages de cette l'architecture	12
2.6.2.	Version 2 : Communication via Queue	13

2.6.2.1. Vue d'ensemble de l'architecture via Queue	14
2.6.2.2. Avantages d'utiliser le module Queue	15
3. Résultat de l'application finale	16
3.1. Connexion	16
3.2. Visualisation	17
4. Conclusion	19

1. Rappel contexte et exigences

1.1. Contexte du projet

Dans le cadre de l'UE Bureau d'étude, l'ingénieur de recherche Gilles ATHIER de LAERO nous a proposé d'écrire un programme dont les techniciens pourront se servir afin de recevoir les informations d'un l'analyseur d'ozone, de les afficher de manières plus pratique, et de pouvoir sauvegarder un fichier au format .csv.

1.2. Objectif et exigences

L'objectif est de développer une application Python pour l'analyseur d'ozone par photométrie UV modèle 49-i du laboratoire d'Aérologie (CNRS). L'application doit établir une connexion avec l'analyseur via port série, Ethernet ou adaptateur USB-série, recevoir et décoder les données en temps réel à un intervalle d'environ 10 secondes, les afficher sous forme de graphiques et les sauvegarder dans un fichier CSV. Les exigences détaillées sont présentées dans la table 1.

Table 1: La liste des exigences. Note: Criticité : Obligatoire (O) / Souhaitée (S)

ID	Criticité	Description
DP001.1	O	Communication série avec l'analyseur
DP001.2	O	Envoi de commandes et réception de données via RS232
DP001.3	S	Envoi de commandes et réception de données via TCP/IP Ethernet
DP002.1	O	Décodage des données reçues
DP002.2	O	Récupération des données exigées
DP003.1	O	Sauvegarde des données dans un fichier CSV
DP004.1	O	Visualisation des données sous forme de graphiques
DP004.2	O	Affichage des données en temps réel
DP004.3	O	Affichage de plusieurs données sur un même graphique
DP004.4	S	Navigation facile entre les différents graphiques

2. Partie technique

2.1. Protocole de communication

Concernant le protocole de communication, deux solutions ont été identifiées : d'une part, le protocole *C-Link*, qui est un protocole propriétaire conçu spécifiquement pour l'analyseur, et d'autre part, le protocole *Modbus*, qui est un standard ouvert et non propriétaire. Nous avons évalué les deux solutions et rencontré des difficultés spécifiques à chaque protocole.

2.1.1. C-link

Nous avons réussi à établir une connexion avec l'analyseur via le port série, à configurer la machine en mode remote et à récupérer les données d'analyse.

2.1.2. Modbus

Après nous être renseigné sur *ModBus*, il nous a semblé plus simple et pratique de nous servir de *C-link*, car nous n'avons pas besoin de faire des appels à plusieurs registres spécifiques pour récupérer les données, avec seulement un nom de fonction *C-link* nous renvoie toutes les données souhaitées.

2.2. Support de connexion

La communication avec l'analyseur peut s'effectuer via différents ports. Nous avons principalement travaillé sur le port série, mais nous avons également exploré des solutions pour le port Ethernet.

2.2.1. Port serie

Nous avons réussi à établir une connexion via le port série de l'analyseur et un adaptateur USB vers l'ordinateur en utilisant la bibliothèque Python `PySerial`. Cette connexion nécessite trois paramètres : le numéro de port (/

dev/ttyUSB0 sous Linux ou COM11 sous Windows), le baudrate (9600) qui définit la vitesse de transmission des données, et l'ID de la machine (49).

La fonction `serial.Serial()` renvoie un objet (par exemple `ser`), permettant d'envoyer et de lire des données à l'aide des méthodes `read()` et `write()` (ex : `ser.write(...)`).

2.2.2. Ethernet

Les tests locaux ont été concluants, cependant la connexion avec l'appareil réel n'a pas pu être établie.

2.3. Communication avec l'analyseur

À l'issue des tests réalisés sur les protocoles de communication, le protocole C-Link a été retenu pour sa simplicité d'utilisation, reposant sur des primitives de lecture et d'écriture de chaînes de caractères.

2.3.1. Syntaxe des requêtes

Chaque requête envoyée à l'analyseur respecte le format suivant :

identifiant + commande + `\r\n`

L'identifiant est obtenu en ajoutant 128 au numéro de l'analyseur et en convertissant le résultat en caractère utf-8. La requête complète est ensuite encodée en séquence d'octets avant d'être transmise via `ser.write()`.

2.3.2. Commandes utilisées

Deux commandes sont utilisées dans le cadre de ce projet :

- `set mode remote` : configure l'analyseur en mode communication à distance. En cas de succès, l'analyseur répond `set mode remote ok`.

- `lrec 1 1` : demande un relevé de données en temps réel. La réponse contient l'ensemble des paramètres mesurés.

2.4. Récupération des données

2.4.1. Forme de la réponse et sauvegarde CSV

Après l'envoi d'une requête, l'analyseur retourne une séquence d'octets lue via `ser.readline()` et décodée en chaîne de caractères avec `.decode('utf-8')`. La réponse à la commande `lrec 1 1` prend la forme suivante :

```
14:14 05-26-26 flags 0C100000 o3 7.469 hio3 0.000 cellai 115685
cellbi 117893 bncht 31.6 lmpt 52.8 o3lt 67.3 flowa 0.751 flowb 0.717
pres 751.8
```

Seules les valeurs utiles sont conservées : date, heure, `o3`, `cellA`, `cellB`, `benchT`, `lampT`, `o3lamp`, `flowA`, `flowB`, `pression`.

Les données sont enregistrées en temps réel dans un fichier CSV, nommé par la date et l'heure de début d'acquisition, et stocké dans le dossier `record/` à la racine du projet.

2.4.2. Désynchronisation

Une difficulté rencontrée est la désynchronisation entre l'envoi de la commande et la réception de la trame de données. Pour y remédier, une fonction vérifie le format de chaque réponse reçue et continue de lire jusqu'à obtenir une trame valide.

2.5. Interface graphique

2.5.1. Exigences

L'application doit être simple d'utilisation et permettre à l'utilisateur de naviguer aisément entre les différents graphiques. Elle doit afficher les données en temps réel, avec la possibilité de comparer certaines données sur un même graphique (*intensitéA*, *intensitéB*). L'utilisateur doit également pouvoir sélectionner le port série utilisé. La figure 1 présente une maquette de l'interface graphique.

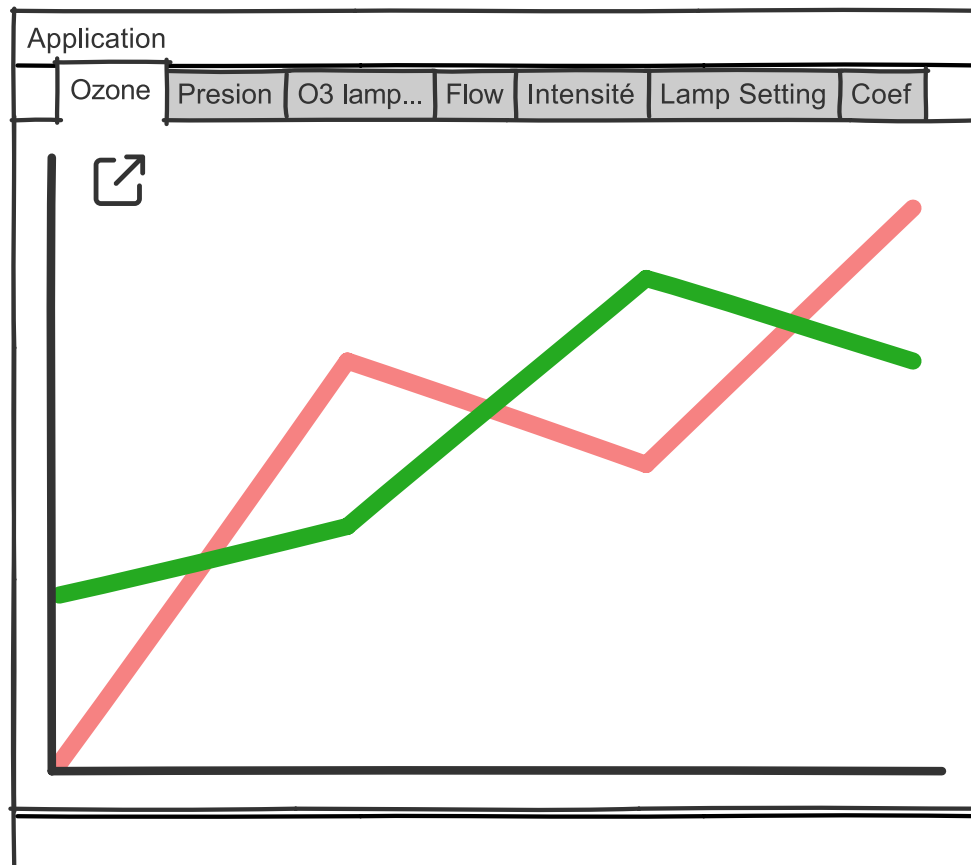


Figure 1: Maquette de la solution de l'application

2.5.2. Choix des bibliothèques

Après plusieurs tests avec la bibliothèque PyQt, la version finale de l'interface utilise Tkinter, pandas et matplotlib. Ces bibliothèques offrent une meilleure flexibilité et une plus grande facilité d'utilisation pour l'affichage de graphiques en temps réel.

2.5.3. Implémentation

L'interface graphique est construite autour de trois modules : `gui.py`, `plots.py` et `components.py`.

La classe `GraphApp` définie dans `gui.py` constitue le cœur de l'interface. Elle crée les onglets de navigation via `ttk.Notebook`, chacun contenant un graphique matplotlib intégré via `FigureCanvasTkAgg`. Le polling de la file d'attente est effectué toutes les 200ms via `root.after()`, garantissant que toutes les opérations sur le `DataFrame` et les graphiques sont réalisées sur le thread principal. Les données sont sauvegardées automatiquement dans le dossier `record/` via `_autosave()` à chaque nouvelle mesure reçue.

Les graphiques sont rendus par deux fonctions définies dans `plots.py` : `create_single_plot()` pour les graphiques à une seule courbe et `create_dual_plot()` pour les graphiques à deux courbes superposées permettant la comparaison directe de données. L'axe des abscisses affiche l'heure réelle au format `HH:MM:SS`.

La classe `Tooltip` définie dans `components.py` affiche une infobulle sous un widget lorsque l'utilisateur le survole, en utilisant les événements Tkinter `<Enter>` et `<Leave>`.

2.6. La communication entre backend et frontend

Le développement s'est déroulé en deux étapes. La première version constitue un prototype permettant de valider les concepts fondamentaux : communication série, parsing des trames et affichage graphique. Cependant, elle présentait des limitations, notamment un risque de race condition ou un manque de scalabilité.

La deuxième version corrige ces limitations en introduisant une architecture producteur-consommateur basée sur une file d'attente (Queue) thread-safe.

2.6.1. Version 1 : Communication via CSV

2.6.1.1. Vue l'ensemble de l'architecture via CSV

La première version repose sur une architecture dans laquelle le thread backend est responsable de la création d'un fichier CSV. À chaque réception d'une ligne de données, ce thread lit, traite et enregistre les données dans le fichier CSV, puis retourne le nom de ce fichier au thread principal. Ce dernier récupère le nom du fichier, l'ouvre et affiche les données dans le graphique. L'architecture est présentée comme dans la figure [2](#).

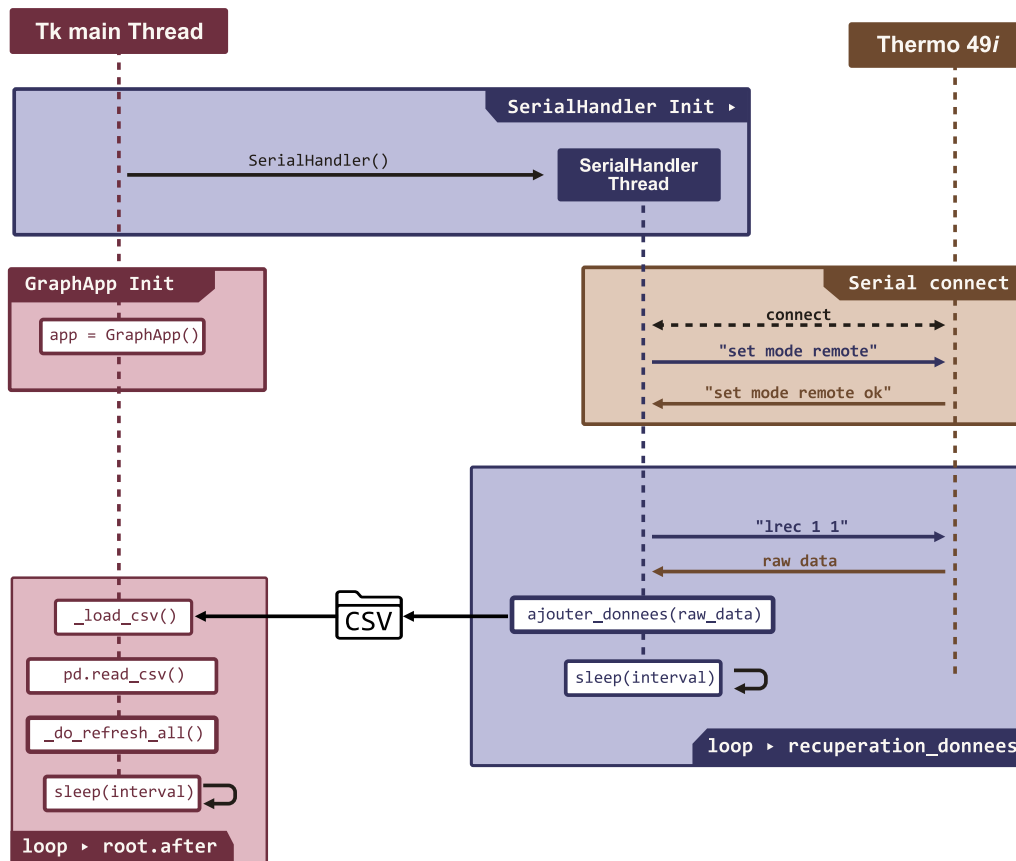


Figure 2: Diagramme de la communication via un fichier csv

2.6.1.2. Des avantages et désavantages de cette l'architecture

L'approche par fichier CSV présente l'avantage d'être simple à implémenter. Le seul canal de communication entre les deux threads est le chemin du fichier CSV, ce qui ne nécessite aucun mécanisme de synchronisation explicite.

Cette approche présente deux limitations principales.

D'une part, en termes de scalabilité, l'interface graphique relit l'intégralité du fichier CSV à chaque cycle de rafraîchissement. Le temps de lecture croît donc linéairement avec le nombre de mesures accumulées, ce qui entraîne

une dégradation progressive des performances lors d'une acquisition longue durée.

D'autre part, en termes de robustesse, une situation de compétition (*race condition*) peut survenir lorsque le thread en arrière-plan écrit une ligne au même moment où le thread principal lit le fichier via `pd.read_csv()`. Ce risque est d'autant plus élevé que le fichier CSV est volumineux, car la lecture d'un fichier de grande taille prend plus de temps, augmentant ainsi la fenêtre temporelle pendant laquelle les deux threads accèdent simultanément au fichier.

2.6.2. Version 2 : Communication via Queue

Afin d'assurer la communication entre le backend et le frontend, une deuxième solution est proposée, ce qui repose sur l'utilisation du module `Queue`.

2.6.2.1. Vue d'ensemble de l'architecture via Queue

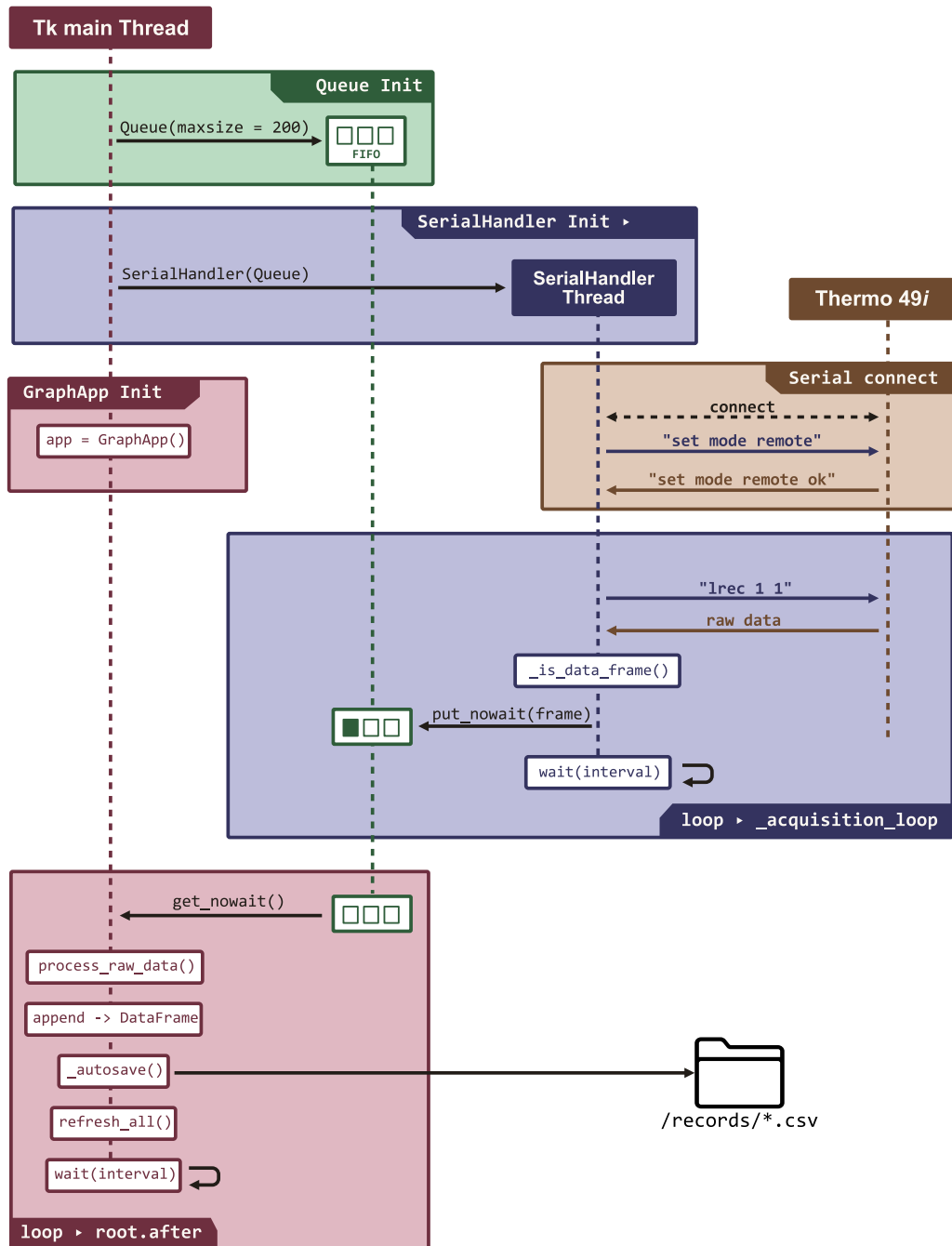


Figure 3: Diagramme de la communication via Queue

La figure 3 illustre l'architecture producteur-consommateur de l'application, organisée autour de quatre acteurs : le thread principal (Tk), le `SerialHandler`, l'analyseur Thermo 49i et la `Queue`.

Au démarrage, `main.py` initialise dans l'ordre : la `Queue` (taille max = 200, FIFO), le `SerialHandler` et l'interface graphique `GraphApp`.

Le `SerialHandler` établit la connexion série avec l'analyseur en envoyant la commande `set mode remote`, et attend la confirmation `set mode remote ok`. Une fois connecté, il démarre son propre thread en arrière-plan via `_acquisition_loop()`. À chaque cycle, il envoie `lrec 1 1` à l'analyseur, valide la trame reçue via `_is_data_frame()`, l'insère dans la `Queue` via `put_nowait()`, puis se met en attente interruptible via `wait(interval)`.

La `Queue` est le canal de communication thread-safe entre le `SerialHandler` et `GraphApp`. Si elle est pleine, la donnée la plus ancienne est supprimée pour laisser place à la plus récente.

`GraphApp` s'exécute sur le thread principal via une boucle `root.after()`. À chaque itération, elle récupère les trames disponibles via `get_nowait()`, les parse via `process_raw_data()`, les ajoute au `DataFrame`, déclenche la sauvegarde automatique via `_autosave()` dans `records/*.csv`, puis redessine les graphiques via `refresh_all()`.

2.6.2.2. Avantages d'utiliser le module `Queue`

L'utilisation du module `Queue` présente plusieurs avantages par rapport à l'approche par fichier CSV. Étant nativement thread-safe, elle ne nécessite aucun verrou manuel (`threading.Lock`), ce qui simplifie la gestion des threads et élimine tout risque de *race condition*. Le backend et le frontend sont complètement découplés : le backend peut continuer à recevoir les données

série sans attendre que l'interface les traite, tandis que les données sont mises en tampon dans la file si le frontend est momentanément lent. Enfin, cette approche évite les accès répétés au disque, le fichier CSV n'étant écrit qu'une seule fois par `_autosave()` sur le thread principal, et non à chaque ligne reçue.

3. Résultat de l'application finale

3.1. Connexion

La figure 4 présente l'écran de connexion de l'application. L'utilisateur doit renseigner quatre paramètres : le port série (ex. COM3 sous Windows ou `/dev/ttyUSB0` sous Linux), le baudrate (9600 par défaut), l'ID de l'analyseur et le délai entre chaque relevé en secondes. L'application propose également un second onglet **Ouvrir un fichier** permettant de charger un fichier CSV enregistré lors d'une session précédente pour visualiser les données hors ligne.

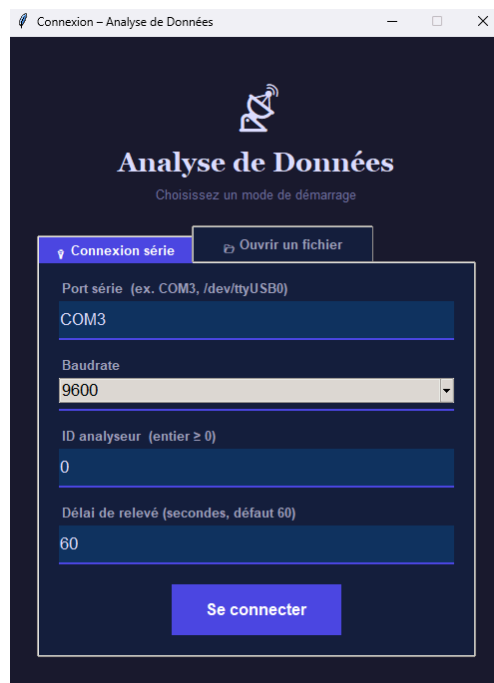


Figure 4: L'écran de démarrage

3.2. Visualisation

L'application affiche les données en temps réel dans des onglets de navigation dédiés à chaque paramètre.

La figure 5 est une capture d'écran de l'application lors d'un test en conditions réelles avec l'analyseur d'ozone. Les valeurs affichées montrent une concentration moyenne d'environ 20 ppb. Lors du test, l'entrée d'air de l'analyseur a été obstruée, ce qui explique la chute de la mesure à 0 ppb. La valeur de -100 ppb, physiquement impossible, est quant à elle attribuée à un bug de l'appareil, confirmé par la trame brute reçue qui contenait effectivement cette valeur anormale.

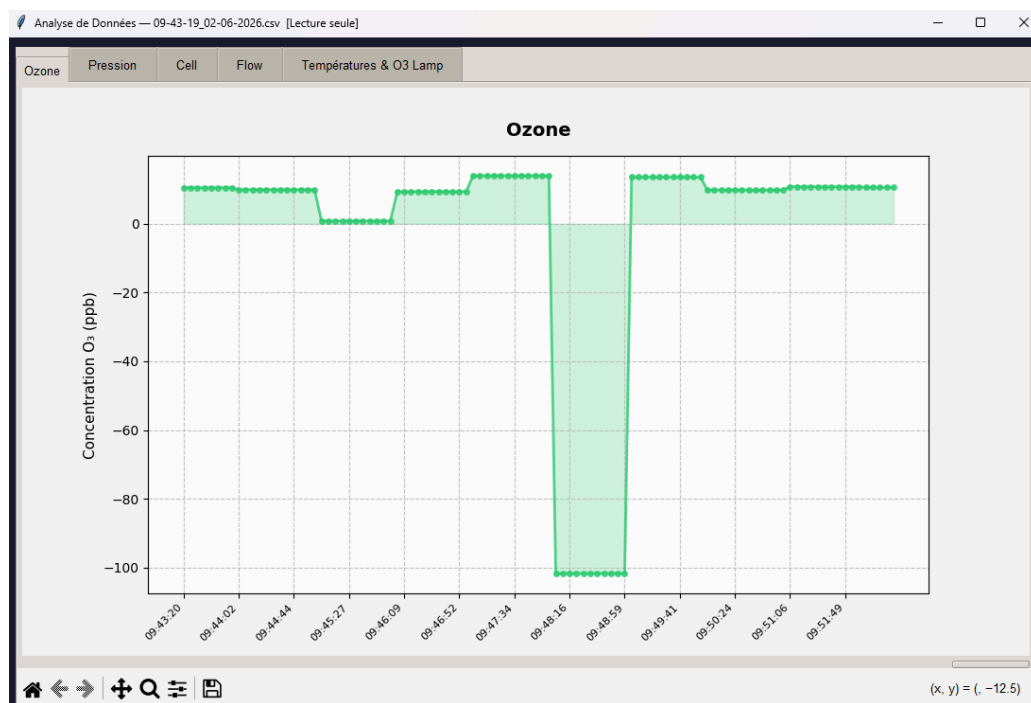


Figure 5: Application finale : affichage d'un paramètre unique

La figure 6 présente une capture d'écran de l'onglet "Températures & O3 Lamp" de l'application, affichant simultanément trois paramètres sur un même graphique : la puissance de la lampe O3 (*o3lamp*), la température du banc (*benchT*) et la température de la lampe (*lampT*). L'affichage multi-courbes permet de visualiser et de comparer l'évolution de ces paramètres en temps réel sur un même axe temporel.

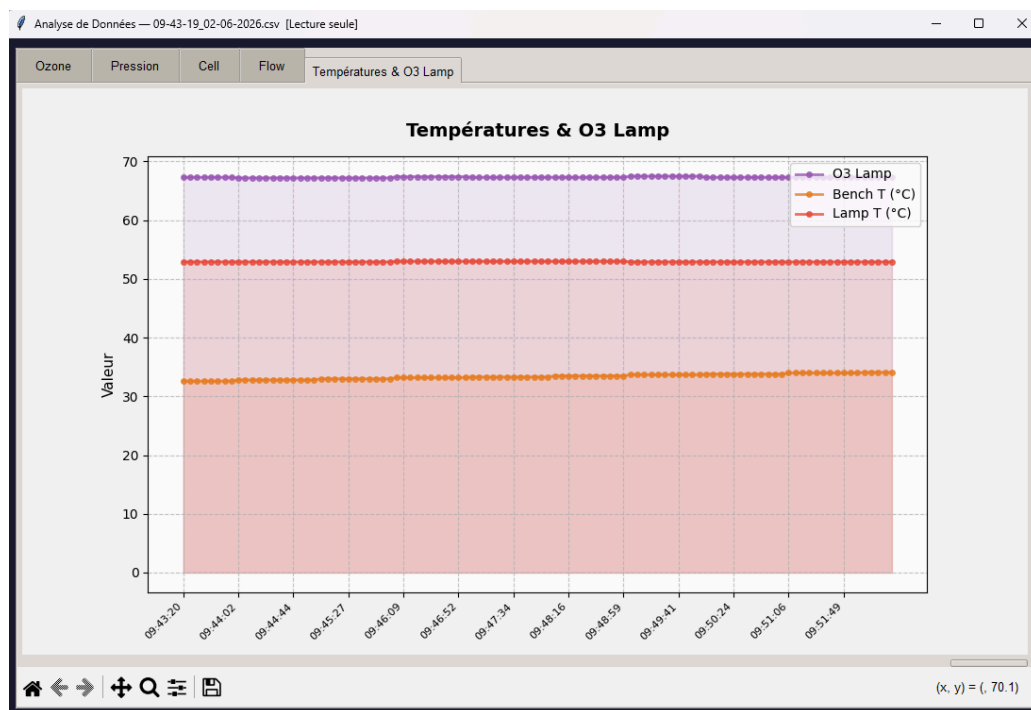


Figure 6: Application finale : affichage simultané de Température et O3 Lamp

4. Conclusion

Ce projet de Bureau d'Études nous a permis de développer une application Python complète pour la communication en temps réel avec l'analyseur d'ozone Thermo Fisher 49i du laboratoire d'Aérologie (CNRS). L'ensemble des exigences obligatoires ont été satisfaites : connexion série, décodage des trames C-Link, affichage graphique en temps réel avec navigation entre les onglets, comparaison de plusieurs paramètres sur un même graphique, et sauvegarde automatique des données dans un fichier CSV.

Le développement en deux versions nous a permis d'identifier et de corriger les limitations architecturales de la première approche, notamment le risque de *race condition* et le manque de scalabilité, au profit d'une architecture producteur-consommateur basée sur le module Queue.

Ce projet nous a également permis de mettre en pratique des concepts fondamentaux de la programmation concurrente, notamment la communication thread-safe entre un thread d'acquisition et une interface graphique en temps réel.